



COMP30026

Models of Computation

Cezary Kaliszyk and William Umboh

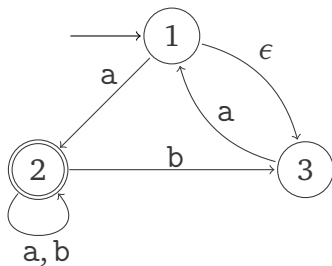
Lecture 13

NFA Determinisation and DFA Minimisation



Reminder: DFAs vs NFAs

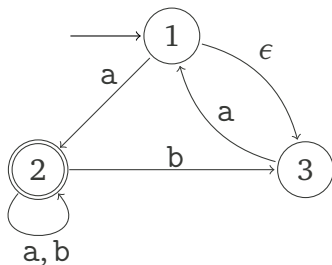
Consider the NFA on the right.
We construct an equivalent DFA.



Reminder: DFAs vs NFAs

Consider the NFA on the right.
We construct an equivalent DFA.

The DFA's start state is $\{1, 3\}$.
From $\{1, 3\}$ a takes us to $\{1, 2, 3\}$.
From $\{1, 2, 3\}$, a takes us back to $\{1, 2, 3\}$, b takes us to $\{2, 3\}$.



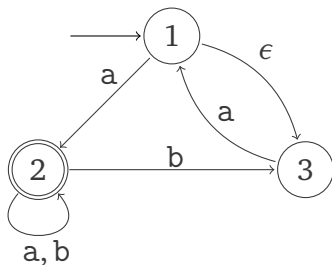
	a	b
$A=\{1,3\}$	B^+	-
$B^+=\{1,2,3\}$	B^+	C^+
$C^+=\{2,3\}$	B^+	C^+

Reminder: DFAs vs NFAs

Consider the NFA on the right.
We construct an equivalent DFA.

The DFA's start state is $\{1, 3\}$.
From $\{1, 3\}$ a takes us to $\{1, 2, 3\}$.
From $\{1, 2, 3\}$, a takes us back to $\{1, 2, 3\}$, b takes us to $\{2, 3\}$.

Any state S which **contains** an accept state from the NFA will be an accept state for the DFA (here marked $^+$).

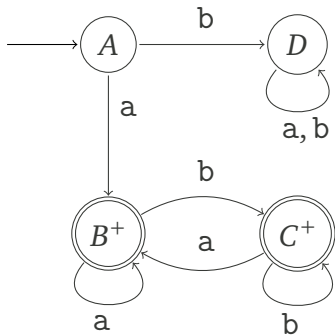


	a	b
$A=\{1,3\}$	B^+	-
$B^+=\{1,2,3\}$	B^+	C^+
$C^+=\{2,3\}$	B^+	C^+

DFAs vs NFAs

Here is the equivalent DFA that we derived.

We add (dead) state D that corresponds to the empty set.



	a	b
$A = \{1, 3\}$	B^+	D
$B^+ = \{1, 2, 3\}$	B^+	C^+
$C^+ = \{2, 3\}$	B^+	C^+
$D = \emptyset$	D	D



The Subset Construction, More Formally

Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA.

For all $R \subseteq Q$, let $E(R)$ be the ϵ closure of R . That is,

$$E(R) = \left\{ q \in Q \left| \begin{array}{l} q \text{ is reachable from some } r \in R \\ \text{by following 0 or more } \epsilon \text{ transi-} \\ \text{tions} \end{array} \right. \right\}.$$

The Subset Construction, More Formally

Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA.

For all $R \subseteq Q$, let $E(R)$ be the ϵ closure of R . That is,

$$E(R) = \left\{ q \in Q \left| \begin{array}{l} q \text{ is reachable from some } r \in R \\ \text{by following 0 or more } \epsilon \text{ transi-} \\ \text{tions} \end{array} \right. \right\}.$$

Let $M = (Q', \Sigma, \delta', q'_0, F')$ be a DFA such that

- $Q' = \mathcal{P}(Q)$,
- $q'_0 = E(\{q_0\})$,
- $F' = \{R \in Q' \mid R \text{ contains an accept state of } N\}$.
- $\delta'(R, a) = \bigcup_{r \in R} E(\delta(r, a))$.

Theorem

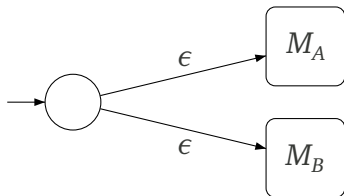
The NFA N is equivalent to the DFA M .

Closure Under Union

Theorem

Let A and B be regular languages. Then $A \cup B$ is regular.

Proof idea: A and B are recognised by some DFAs M_A and M_B , respectively. Then this NFA recognises $A \cup B$:



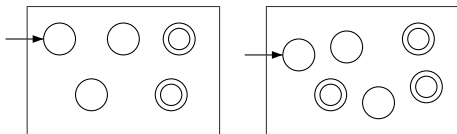
The ϵ -transitions go to the start states of M_A and M_B .

Closure Under Concatenation

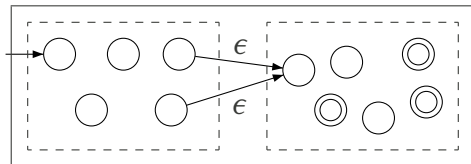
Theorem

Let A and B be regular languages. Then $A \circ B$ is regular.

Proof idea: Suppose these are DFAs for A and B :



Then this NFA recognises $A \circ B$:





That Last Construction, More Formally

Let recognisers for A and B be these DFAs, respectively, where Q and Q' are disjoint:

- $M_A = (Q, \Sigma, \delta, q_0, F)$
- $M_B = (Q', \Sigma, \delta', q'_0, F')$

That Last Construction, More Formally

Let recognisers for A and B be these DFAs, respectively, where Q and Q' are disjoint:

- $M_A = (Q, \Sigma, \delta, q_0, F)$
- $M_B = (Q', \Sigma, \delta', q'_0, F')$

Let N be the NFA $(Q \cup Q', \Sigma, \delta'', \{q_0\}, F')$, where

$$\delta''(q, v) = \begin{cases} \{\delta'(q, v)\} & \text{if } q \in Q' \text{ and } v \in \Sigma \\ \{\delta(q, v)\} & \text{if } q \in Q \text{ and } v \in \Sigma \\ \{q'_0\} & \text{if } q \in F \text{ and } v = \epsilon \end{cases}$$

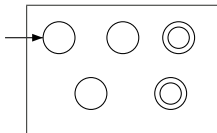
Then the NFA N recognises the language $A \circ B$.

Closure Under Kleene Star

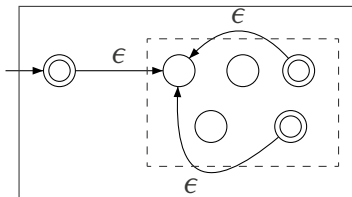
Theorem

Let A be a regular language. Then A^ is a regular language.*

Proof idea: Suppose this DFA recognises A :



Here is how we construct an NFA to recognise A^* :



Closure Results for Regular Languages

Regular languages have more closure properties.

Theorem

Let A and B be regular languages over an alphabet Σ . Then, the following languages are also regular:

- $A \cap B$,
- A^c (the complement of A , that is, $\Sigma^* \setminus A$),
- $A \setminus B$ (follows from the first two, since $A \setminus B = A \cap B^c$),
- the reversal of A .

Corollary

The regular languages are closed under Boolean combination.



Algorithms for Manipulating Automata

We will see some of these algorithms in the tutorials.

Bonus: Think how to formally define a DFA for $A \cap B$.



Equivalence of DFAs

Are two DFAs equivalent?

Are languages the same?



Equivalence of DFAs

Are two DFAs equivalent?

Are languages the same?

What about minimal DFAs?

- Smallest number of states...



Equivalence of DFAs

Are two DFAs equivalent?

Are languages the same?

What about minimal DFAs?

- Smallest number of states...
- Unique starting state...

Equivalence of DFAs

Are two DFAs equivalent?

Are languages the same?

What about minimal DFAs?

- Smallest number of states...
- Unique starting state...
- Total and deterministic transitions
- Same accepting states.



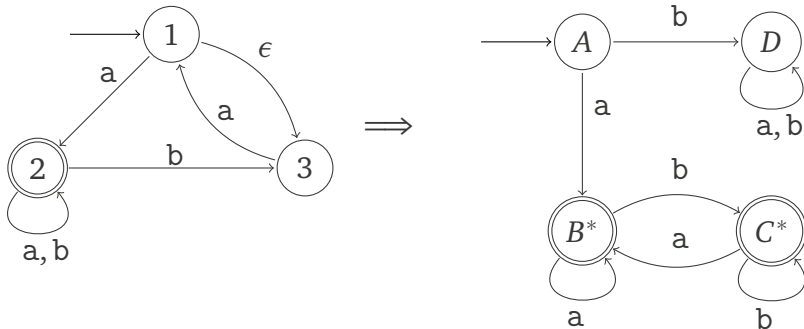
Minimizing DFAs

Equivalent DFAs can have different sizes. Example?

Minimizing DFAs

Equivalent DFAs can have different sizes. Example?

Algorithms (e.g. subset construction) often give not minimal.



$A = \{1, 3\}$, $B^* = \{1, 2, 3\}$, $C^* = \{2, 3\}$, and $D = \emptyset$.



Generating a Minimal DFA

Algorithm that given an NFA produces an equivalent **minimal** DFA. (Input can also be a DFA).

1. Reverse the NFA;
2. Determinize the result;
3. Reverse again;
4. Determinize.

Generating a Minimal DFA

Algorithm that given an NFA produces an equivalent **minimal** DFA. (Input can also be a DFA).

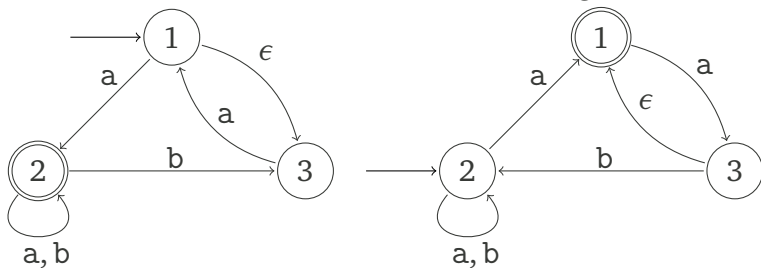
1. Reverse the NFA;
2. Determinize the result;
3. Reverse again;
4. Determinize.

To reverse NFA A with initial state q_0 and accept states $F \neq \emptyset$:

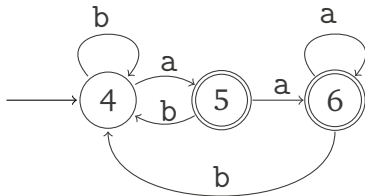
1. If $F = \{q\}$, let q be the accept state.
2. Otherwise add a new state q which becomes the only accept state; then, for each state in F , add an ϵ transition to q .
3. Reverse every transition in the resulting NFA, making q the initial state and q_0 the (only) accept state.

Minimization Example

Consider again the NFA that we determinized two slides ago. Here it is on the left, with its reversal on the right:

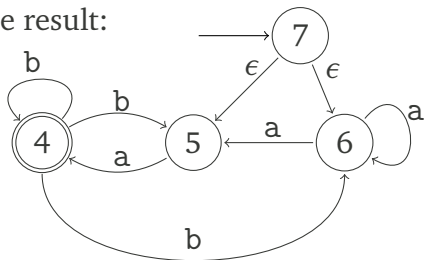


Now make the reversed NFA deterministic (we have renamed the states to avoid later confusion: 4 corresponds to {2}, 5 to {1, 2}, and 6 to {1, 2, 3}).

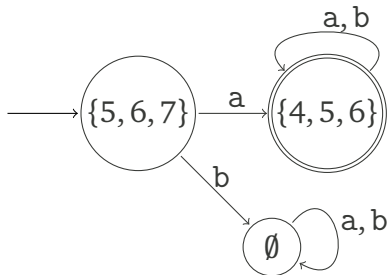


Minimization Example

Now reverse the result:



Finally make the result deterministic:





Pattern Matching

Important for

- Search engines (code search)
- Lexical analysis of programs
- Scripting languages (Perl) and tools (grep, awk, sed)
- DNA analysis



Pattern Matching

Important for

- Search engines (code search)
- Lexical analysis of programs
- Scripting languages (Perl) and tools (grep, awk, sed)
- DNA analysis

Finite automata for pattern matching

- How would you recognize the word “bug”?



Regular Expressions

- Compact notation for describing regular languages.
- Restricted patterns
- See also “regexes” in JavaScript or Python.

Example: $(0 \cup 1)(0 \cup 1)((0 \cup 1)(0 \cup 1))^*$

In Python: `r"(0|1)(0|1)((0|1)(0|1))*"`



Formal Syntax

The **regular expressions** over an alphabet $\Sigma = \{a_1, \dots, a_n\}$ are given by the grammar

$$\begin{array}{lcl} \text{regex}p & \rightarrow & a_1 \quad | \quad \dots \quad | \quad a_n \quad | \quad \epsilon \quad | \quad \emptyset \\ & & | \quad \text{regex}p \cup \text{regex}p \quad | \quad \text{regex}p \circ \text{regex}p \quad | \quad \text{regex}p^* \end{array}$$



Formal Syntax

The **regular expressions** over an alphabet $\Sigma = \{a_1, \dots, a_n\}$ are given by the grammar

$$\begin{array}{lcl} \text{regex} & \rightarrow & a_1 \mid \dots \mid a_n \mid \epsilon \mid \emptyset \\ & & \mid \text{regex} \cup \text{regex} \mid \text{regex} \circ \text{regex} \mid \text{regex}^* \end{array}$$

Semantics:

$$\begin{array}{ll} L(a) & = \{a\} \\ L(\epsilon) & = \{\epsilon\} \\ L(\emptyset) & = \emptyset \\ L(R_1 \cup R_2) & = L(R_1) \cup L(R_2) \\ L(R_1 R_2) & = L(R_1) \circ L(R_2) \\ L(R^*) & = L(R)^* \end{array}$$



Notational Conveniences

Can omit $\circ$ and sometimes parentheses.

Binding precedence: star $>$ concatenation $>$ union.



Notational Conveniences

Can omit $\circ$ and sometimes parentheses.

Binding precedence: star $>$ concatenation $>$ union.

Examples:

1. ab means $(a \circ b)$
2. ab^* means $(a \circ (b^*))$.
3. $a \cup bc^*$ means $(a \cup (b \circ (c^*)))$.



Regular Expressions – Examples

ϵ :

1 :

110 :

$((0 \cup 1)(0 \cup 1))^*$:

$(0 \cup \epsilon)(\epsilon \cup 1)$:

1^* :

$\epsilon \cup 1 \cup (\epsilon \cup 1)^*(\epsilon \cup 1)$:

$(1^*0^*)^*$:



Regular Expressions – Examples

ϵ : $\{\epsilon\}$

1 : $\{1\}$

110 : $\{110\}$

$((0 \cup 1)(0 \cup 1))^*$: all binary strings of even length

$(0 \cup \epsilon)(\epsilon \cup 1)$: $\{\epsilon, 0, 1, 01\}$

1^* : all finite sequences of 1s

$\epsilon \cup 1 \cup (\epsilon \cup 1)^*(\epsilon \cup 1)$: all finite sequences of 1s

$(1^*0^*)^*$: Σ^*



Regular Expressions to NFAs

Theorem

A language is regular iff it can be described by a regular expression.

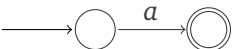
Regular Expressions to NFAs

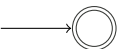
Theorem

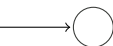
A language is regular iff it can be described by a regular expression.

Proof idea ($\Leftarrow$): Construct NFA from regular expression R .
Bottom up. (Formally: induction).

Base cases: $R = a \in \Sigma$, $R = \epsilon$, or $R = \emptyset$.

If $R = a$: Construct 

If $R = \epsilon$: Construct 

If $R = \emptyset$: Construct 

Inductive step: $R = R_1 \cup R_2$, $R = R_1 \circ R_2$, or $R = R_1^*$. Use the constructions for closure under regular operations.